



SERVERLESS AQI PREDICTION SYSTEM

Islamabad Air Quality Forecasting
Internship Project Report

Prepared by

Wali Muhammad Nasir

10Pearls — Shine Internship Program

Project: AQI Predictor — Serverless AQI Forecasting System

Live Dashboard: isb-aqi-predictor.streamlit.app

Table of Contents

1. Executive Summary	4
2. System Overview	4
3. Data Sourcing.....	5
3.1 Why Open-Meteo, not OpenWeather or AQICN	5
3.2 Data provenance and its limitations.....	5
3.3 Historical data window	5
3.4 Multi-city extensibility	5
4. Exploratory Data Analysis — Key Findings	6
4.1 PM2.5 is the strongest single hourly predictor of AQI	6
4.2 A counter-intuitive seasonal pattern, and its explanation.....	6
4.3 Weather variables genuinely matter for a multi-day forecast	6
4.4 Outliers were retained, not removed	6
5. Feature Engineering.....	7
5.1 Feature categories.....	7
5.2 Chronological train/test split.....	7
5.3 A feature dropped on evidence, not assumption	7
6. Feature Store & Model Registry (Hopsworks)	7
6.1 Engineering challenges encountered and resolved.....	7
7. Model Training and Selection.....	8
7.1 Models compared	8
7.2 Why performance degrades with forecast horizon	9
7.3 A per-horizon model selection, based on evidence	9
7.4 Deep learning (LSTM)	9
8. Model Explainability (SHAP)	9
8.1 Near-term forecasts are momentum-driven.....	9
8.2 Longer-term forecasts shift toward weather and seasonal signals.....	10
9. Hazardous AQI Alerts	10
10. Dashboard.....	10
10.1 A deployment-specific compatibility issue	10
10.2 Keeping the dashboard aligned with the latest trained model.....	11
11. Automation (CI/CD)	11
11.1 Problems encountered and resolved	11
12. Limitations and Future Work.....	12

13. Conclusion..... 12

1. Executive Summary

This project delivers an end-to-end, serverless system that forecasts Islamabad's Air Quality Index (AQI) 24, 48, and 72 hours ahead, using historical and live weather and pollutant data. The system covers the full machine learning lifecycle specified in the project brief: automated hourly data collection, a cloud-based feature store, model training and comparison, a versioned model registry, an interactive dashboard, and fully automated CI/CD pipelines — all built without provisioning or managing a dedicated server.

Rather than treating this as a checklist exercise, each stage involved a genuine design decision with trade-offs, and several stages surfaced real engineering problems that had to be diagnosed and resolved rather than worked around. This report documents those decisions and problems, and the reasoning behind how each was resolved, alongside the final system's architecture and results.

Live dashboard: isb-aqi-predictor.streamlit.app (Streamlit Community Cloud) — updates hourly from an automated data pipeline running on GitHub Actions.

2. System Overview

The final system consists of five components, each running independently and communicating only through the Hopsworks Feature Store and Model Registry:

- Backfill pipeline — one-time historical data collection (2.5 years, hourly).
- Hourly feature pipeline — live data collection, automated via GitHub Actions.
- Daily training pipeline — retrains and re-registers models, automated via GitHub Actions.
- Feature Store & Model Registry (Hopsworks) — centralized, versioned storage for both data and models.
- Dashboard (Streamlit, deployed to Streamlit Community Cloud) — loads the latest models and data, displays a 3-day forecast with hazardous-AQI alerts.

No component requires a persistent server that the team manages directly: GitHub Actions runners are ephemeral, Hopsworks and Streamlit Cloud are both managed platforms. This satisfies the brief's core requirement that feature generation, training, storage, and scheduling be serverless.

3. Data Sourcing

3.1 Why Open-Meteo, not OpenWeather or AQICN

The project brief named OpenWeather and AQICN as example data sources. Both were evaluated and ultimately rejected in favour of Open-Meteo, for concrete, practical reasons rather than preference:

OpenWeather: the free tier only provides current weather and pollution data. Genuine historical data (required to build a training set) sits behind the paid "One Call 3.0" tier, which requires a credit card on file. This was judged an unnecessary constraint given a free, equally capable alternative existed.

AQICN: the live/current API is free and usable, but genuine historical data access requires submitting a manual request form and waiting for a human response — incompatible with a fully automated, programmatic backfill pipeline.

Open-Meteo (chosen): provides both historical (from 1940, via ERA5/ERA5-Land reanalysis) and live weather and air quality data from a single provider, with no API key, no credit card, and a free non-commercial tier of up to 10,000 calls/day. Using one provider for both historical and live data also avoided a subtler risk: mixing two providers with different measurement methods, units, or station sources could have introduced inconsistencies between training data and live data.

3.2 Data provenance and its limitations

Open-Meteo's historical data is reanalysis data (ERA5 + CAMS) — a scientifically-produced model estimate that blends real observations with atmospheric physics models, rather than a direct single-station sensor reading. A production system would ideally cross-validate against Pakistan's local monitoring stations where available.

3.3 Historical data window

The backfill covers roughly 2.5 years of hourly data (2024-01-01 onward, later extended to close a data gap — see Section 11.1). At least one full seasonal cycle is required for the model to learn Islamabad's seasonal AQI patterns at all; more than one cycle allows the model to distinguish a normal seasonal pattern from a one-off anomaly, and supports a cleaner chronological train/test split (see Section 5).

3.4 Multi-city extensibility

Although the deployed system covers Islamabad only, both the backfill and live pipelines are structured around a city dictionary keyed by latitude/longitude rather than hardcoded coordinates. Adding a second city is a one-line configuration change, not a rewrite — a deliberate design decision made early, at negligible cost, to keep the architecture genuinely extensible.

4. Exploratory Data Analysis — Key Findings

EDA was not treated as a formality; several findings materially shaped later feature engineering and modelling decisions.

4.1 PM2.5 is the strongest single hourly predictor of AQI

Across the full dataset, PM2.5 showed the strongest linear correlation with US AQI ($r = 0.57$), followed by PM10 ($r = 0.49$). This matches the AQI formula's design, which weights PM2.5 heavily due to its deeper health impact (fine enough to penetrate lung tissue and enter the bloodstream).

4.2 A counter-intuitive seasonal pattern, and its explanation

AQI in the dataset rises in summer and falls in winter — the opposite of the commonly assumed "winter smog" pattern associated with crop-burning in the Punjab plains. Investigating this: a monthly-average plot showed ozone tracking the seasonal AQI wave closely, while PM2.5 moved in the opposite direction (higher in winter). Correlating temperature against both pollutants explained the mechanism: temperature correlates strongly positively with ozone ($r = 0.64$, since ozone forms photochemically in heat and sunlight) and strongly negatively with PM2.5 ($r = -0.30$, consistent with winter temperature-inversion trapping of particulates). Islamabad's overall AQI is therefore not driven by one dominant pollutant, but by a seasonal hand-off between two different pollution mechanisms — a finding that justified keeping all six pollutants, plus month/seasonal features, in the model rather than relying on a single "master pollutant."

4.3 Weather variables genuinely matter for a multi-day forecast

It was initially unclear whether weather data was necessary for an "AQI predictor." This was resolved by direct physical reasoning, later confirmed by SHAP (Section 8): today's AQI has weak predictive power over AQI three days out on its own, but weather variables are the primary mechanism by which future pollution levels change — wind disperses pollutants, temperature drives ozone formation and PM2.5 accumulation, humidity and pressure relate to atmospheric stagnation. Weather correlations with PM2.5 (humidity +0.32, wind speed -0.19, temperature -0.30) were all directionally consistent with known atmospheric science, supporting confidence in the data.

4.4 Outliers were retained, not removed

Box plots showed frequent high-side outliers in PM2.5, PM10, and CO — interpreted as genuine pollution spike events rather than sensor error, since no physically impossible values (e.g. negative concentrations) were present. These outliers were deliberately kept in the dataset, since detecting exactly this kind of spike is central to the system's purpose.

5. Feature Engineering

5.1 Feature categories

- Time-based: hour, month, day-of-week (day-of-month was excluded — see Section 5.3).
- Lag features: AQI at 1h, 3h, 6h, and 24h prior — giving the model recent momentum/history, since each row is otherwise evaluated independently.
- AQI change rate (1h, 24h): captures direction and speed of change, not just a snapshot value.
- Future targets: AQI at +24h, +48h, +72h — the model's actual prediction targets, computed on historical data via a negative time-shift, since the true future outcome is already known for past rows.

5.2 Chronological train/test split

The data was split chronologically (train on the earlier ~2 years, test on the most recent ~6 months) rather than with a random shuffle. This was a deliberate choice: a random split would let the model train on data that is chronologically after some of its test data, producing misleadingly optimistic accuracy that would not hold up against genuinely new, unseen future data — exactly the situation the model faces once deployed.

5.3 A feature dropped on evidence, not assumption

Day-of-month was excluded as an input feature. Rather than dropping it purely on intuition, this was later corroborated by SHAP analysis (Section 8), which showed several weather and short-lag features contributing minimally to predictions at the 24h horizon — reinforcing that feature inclusion decisions should be evidence-based rather than assumption-based wherever practical.

6. Feature Store & Model Registry (Hopsworks)

Hopsworks was chosen over Vertex AI (the brief's other suggested option) as it offered a genuinely free, unrestricted serverless tier suited to a project of this scale, and because working with a dedicated feature-store platform — rather than ad hoc file storage — was judged a stronger, more realistic demonstration of an actual production ML architecture.

6.1 Engineering challenges encountered and resolved

Setting up Hopsworks on a Windows development machine surfaced a sequence of real, non-trivial environment issues:

- Installing the hopsworks package failed compiling a transitive dependency (twofish), which requires a C compiler. Resolved by installing Microsoft's Visual C++ Build Tools — a one-time environment fix, not a flaw in the chosen package.
- `hopsworks.login()` failed with a missing `/tmp` directory — the client assumes a Unix-style path that does not exist on Windows by default. Resolved by creating the directory programmatically at runtime.
- Feature Group creation failed under the default Delta Lake storage format, which itself depends on Apache Spark and, in turn, typically a local Java runtime and Windows-specific Hadoop compatibility shims. The Feature Group was instead created using Hopsworks' alternative native format (Hudi), avoiding that dependency chain entirely — a deliberate scope decision.
- Reading data back from the Feature Store intermittently failed with server-side errors from Hopsworks' Arrow Flight query service. This recurred across multiple, unrelated scripts over several days, confirming it as a genuine platform-side reliability characteristic. A fallback read method (Hive-based querying) was added everywhere the Feature Store is read, so a transient failure retries automatically instead of crashing the calling script.

Notably, when the same requirements.txt was installed on GitHub Actions' Linux runners, none of the Windows-specific issues (compiler tools, the `/tmp` path) occurred — confirming those were genuinely platform-specific development friction, not problems with the chosen tools themselves.

7. Model Training and Selection

7.1 Models compared

Ridge Regression, Random Forest, and XGBoost were trained and evaluated at all three forecast horizons, using RMSE, MAE, and R^2 on the held-out chronological test set:

Horizon	Model	RMSE	R^2	Notes
24h	XGBoost (selected)	14.52	0.748	Best performer at the strongest-signal horizon; outperformed Ridge and Random Forest.
48h	Ridge Regression (selected)	20.13	0.510	Matched or slightly outperformed tree-based models as predictive signal weakened.
72h	Ridge Regression (selected)	22.60	0.383	Same reasoning as 48h; Random Forest underperformed both at every horizon.

7.2 Why performance degrades with forecast horizon

R^2 fell from 0.748 (24h) to 0.510 (48h) to 0.383 (72h) across all model types. This is an expected, structurally sound result rather than a weakness: uncertainty compounds over a forecasting horizon in any time-series system, and the model's strongest features (short-term AQI lags) are naturally most informative about the near future and progressively less informative further out. A model showing identical accuracy at all three horizons would have been the more suspicious result.

7.3 A per-horizon model selection, based on evidence

XGBoost outperformed at the 24h horizon, where the signal is strongest and most exploitable by a non-linear model. Ridge Regression matched or slightly outperformed both tree-based models at 48h and 72h, where the signal is weaker and added model complexity showed diminishing or negative returns. Rather than deploying one model type across all horizons for simplicity, the final system uses XGBoost for the 24h forecast and Ridge Regression for the 48h and 72h forecasts — a decision made on the comparison evidence above, not by default preference for either a simple or complex model.

7.4 Deep learning (LSTM)

An LSTM implementation was scoped (sequence construction, feature scaling) but not carried through to a full trained comparison, given the evidence already gathered: Ridge Regression's strong relative performance indicated a largely linear-dominant signal once lag and change-rate features were engineered, and the dataset's size (~18,000 training rows) is modest for the class of problem where deep sequence models typically show their clearest advantage. This is flagged explicitly as a limitation and an item for future work, rather than presented as a completed comparison.

8. Model Explainability (SHAP)

SHAP (SHapley Additive exPlanations) was used to determine which features actually drive each model's predictions, both to validate the models' reasoning and to directly test the hypotheses formed during EDA.

8.1 Near-term forecasts are momentum-driven

At the 24h horizon, `aqi_lag_1h` and `aqi_change_rate_1h` were by far the most influential features, followed by `pm2_5` and `month`. This confirms the 24h model is primarily reasoning from very recent AQI momentum — a sensible strategy at short range.

8.2 Longer-term forecasts shift toward weather and seasonal signals

At 48h and 72h, `aqi_lag_24h` (a same-time-yesterday comparison) overtook `aqi_lag_1h` in importance, and weather features — dew point, relative humidity, temperature, surface pressure — rose sharply in importance, while raw pollutant levels faded in relative importance. This is direct, model-level evidence for the hypothesis formed during EDA (Section 4.3): near-term AQI forecasting leans on recent momentum, while longer-range forecasting depends more on weather-driven, seasonal dynamics. This is treated as one of the project's more substantive analytical findings, since it was derived from the trained models' actual behaviour rather than assumed in advance.

9. Hazardous AQI Alerts

Forecasted AQI values are mapped to the standard US EPA AQI categories (Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, Hazardous). An alert is raised for any forecasted horizon at AQI 151 or above ("Unhealthy") — the threshold at which health guidance typically recommends limiting outdoor exposure for the general population, not only sensitive groups. Each alert level is paired with a short, category-appropriate health recommendation shown on the dashboard.

10. Dashboard

The dashboard is built in Streamlit and deployed to Streamlit Community Cloud, connecting directly to the Hopsworks Feature Store and Model Registry rather than to any static or local data. It presents:

- Current AQI and current weather conditions.
- A 3-day forecast with colour-coded AQI category cards and health guidance.
- A recent (72-hour) trend chart and a full historical daily-average trend.
- Current pollutant-level breakdown.
- Hazardous AQI alerts, where applicable.
- SHAP feature-importance plots for all three models, for transparency into model reasoning.

10.1 A deployment-specific compatibility issue

The initial Streamlit Cloud deployment failed with `ModuleNotFoundError: No module named 'imp'`. Streamlit Cloud defaulted to Python 3.14, from which the `imp` module was fully removed; the installed version of Hopsworks' packaging still referenced it. This was resolved by pinning the deployment to Python 3.11 via a `runtime.txt` file, matching the local development environment where the issue did not occur — a reminder that dependency compatibility can depend on the Python interpreter version itself, not only on package versions.

10.2 Keeping the dashboard aligned with the latest trained model

The dashboard initially loaded a hardcoded Model Registry version, so it kept showing the same forecast and accuracy figures even as the daily training pipeline produced newer, updated model versions in the background. This was resolved by having the dashboard query the registry for the highest available version number at load time, so it always reflects the most recently trained model without requiring a code change after each retraining cycle.

11. Automation (CI/CD)

Both pipelines specified in the brief were implemented using GitHub Actions rather than Apache Airflow, since Airflow requires a persistently running scheduler/webserver process — at odds with the project's serverless requirement — whereas GitHub Actions runs entirely on ephemeral, managed runners.

- Hourly feature pipeline: fetches live weather and air-quality data, computes features (including lags, using recent Feature Store history), and inserts a new row into the Feature Store.
- Daily training pipeline: reads the current Feature Store contents, retrains all three models on a fresh chronological split, and registers new model versions to the Model Registry (implementing the brief's unconditional daily-retraining requirement, rather than a more complex drift-triggered scheme — see Section 12).

11.1 Problems encountered and resolved

Automating the live pipeline surfaced a chain of genuine, production-style bugs, each diagnosed and fixed in turn rather than worked around:

- A timezone inconsistency between the live weather and live air-quality API calls caused the two datasets to merge into zero rows, since one call defaulted to local time and the other to UTC. Resolved by keeping both calls consistently in UTC, matching the historical backfill.
- Schema-compatibility errors on insert (integer type mismatches, and missing target columns, which genuinely cannot exist for a live row since they depend on future data). Resolved with explicit type casting and by inserting null targets for live rows, since the trained model's output — not a stored target column — is what stands in for the unknown future value during live prediction.
- GitHub Actions' free-tier scheduled workflows do not run reliably on exact hourly boundaries — observed run times were irregular, a documented platform limitation rather than a configuration error. This broke the original lag-feature lookup, which required an exact timestamp match 24 hours prior and consequently failed on almost every live row, causing the dashboard to silently fall back to stale historical data for several days without erroring.

- This was resolved by changing the lag lookup to a nearest-available-timestamp match within a tolerance window (2 hours), rather than requiring an exact match — a deliberate adaptation to the platform's real scheduling behaviour rather than an attempt to force more precise scheduling that the free tier does not support.
- Fixing the above surfaced two further issues in sequence: a timezone-awareness mismatch between live-built timestamps (naive) and timestamps read back from Hopsworks (UTC-aware), and a type mismatch where lag values that happened to be whole numbers were inferred as integers rather than the required floating-point schema type. Both were resolved with explicit timezone localisation and type casting.

This debugging sequence — four compounding issues surfaced by a single fix attempt — is treated in this report as a legitimate finding in its own right: it demonstrates the gap between code that works against carefully controlled local test data and code that must run correctly, unattended, against a real external platform's actual (imperfect) scheduling and service behaviour.

12. Limitations and Future Work

- Deep learning comparison (LSTM) was scoped but not completed to a full trained comparison against Ridge/XGBoost (Section 7.4).
- The system currently covers a single city (Islamabad). The architecture — a city configuration dictionary keyed by coordinates, used consistently across the backfill and live pipelines — was deliberately built to make adding further cities a small, low-risk change rather than a redesign, though this was not carried out within the current scope.
- Historical and live data both derive from a single reanalysis-based provider (Open-Meteo/CAMS). A production deployment would ideally cross-validate against a local physical monitoring network where one exists (see Section 3.2).

13. Conclusion

The completed system satisfies every component specified in the project brief: a serverless feature pipeline, historical backfill, a training pipeline comparing multiple model classes with tracked metrics, a versioned model registry, fully automated hourly and daily CI/CD, an interactive dashboard with hazardous-AQI alerting, and SHAP-based explainability. Beyond simply meeting these requirements, the project surfaced and resolved a series of genuine engineering problems — spanning data-source selection, cross-platform environment issues, model-selection trade-offs, and live automation reliability — each of which is documented above with the reasoning behind how it was resolved, rather than simply the fact that it was.